

Intern Project Description Sheets

PROJECT 1 — Security Testing of a Multi-Agent AI System

Intern Role: AI Security Tester **Duration:** 2 Months

Problem Statement

AI agents are being used more and more in real applications — they read data, call tools, talk to each other, and take actions. But most of these systems are never properly tested for security. A bad actor can manipulate an AI agent through cleverly crafted inputs, bypass its instructions, steal information it has access to, or abuse the tools it can call. The goal of this project is to build a simple multi-agent AI system and then attack it — finding every security weakness you can and documenting it clearly.

What You Will Build

Phase 1 — Build the Target System (3 weeks)

Build a simple multi-agent system with at least 3 agents talking to each other. A suggested setup:

- **Orchestrator Agent** — takes a user question and decides which sub-agent to call
- **Research Agent** — searches or reads documents to answer questions
- **Action Agent** — performs a task like writing a file, sending a mock email, or querying a database

This system does not need to be complex. It just needs to be real enough to test. Use any framework you are comfortable with (LangGraph, CrewAI, AutoGen, or plain Python with API calls).

Phase 2 — Security Testing (5 weeks)

Test the system across the following attack categories:

Attack Type	What It Means
Prompt Injection	Hiding malicious instructions inside user input to hijack the agent
Indirect Prompt Injection	Injecting instructions through external data the agent reads (documents, search results)
Agent Hijacking	Tricking one agent into overriding another agent's instructions
Tool Misuse	Getting an agent to call a tool it should not call, or with wrong parameters
Data Leakage	Getting an agent to reveal its system prompt, memory, or another user's data
Denial of Service	Inputs that cause the agent to loop, hang, or consume excessive API calls

Attack Type	What It Means
Privilege Escalation	Making a low-trust agent perform actions only a high-trust agent should do

For each attack, you must run at least 10 test cases and record the result.

Deliverables

1. Codebase

- GitHub repository with the multi-agent system code
- A `README.md` explaining how to run the system locally (step-by-step, plain English)

2. Test Case Log (Excel or Google Sheet)

Each row in the sheet must have:

Column	What to Fill
Attack Type	e.g. Prompt Injection
Test Input	The exact input or payload used
Expected Behaviour	What the agent should have done
Actual Behaviour	What the agent actually did
Result	PASS (safe) or FAIL (vulnerable)
Severity	Low / Medium / High / Critical
Notes	Any extra observation

3. Final Security Report (PDF, max 15 pages)

Structure it exactly like this:

- Executive Summary (1 page — what you tested, how many vulnerabilities found, overall risk level)
- System Description (1 page — explain the agents you built)
- Attack Results by Category (one section per attack type — what you tried, what worked, what didn't)
- Top 5 Most Critical Vulnerabilities Found (with screenshot or copy of the exact input that caused it)
- Recommendations (what should be fixed and how)

Evaluation Benchmark

Criteria	Target
Number of attack types covered	All 7 listed above
Minimum test cases	70 total (10 per attack type)
Vulnerabilities found and documented	At least 5 real vulnerabilities

Criteria	Target
Report clarity	A non-technical person should understand the executive summary
System runs locally	Must run with one command after setup

PROJECT 2 — Production Deployment of a Multi-Agent System

Intern Role: MLOps / DevOps Engineer **Duration:** 2 Months

Problem Statement

Building an AI agent on your laptop is easy. Making it run reliably in a real production environment — where it needs to handle multiple users, recover from crashes, be monitored, and be deployable on any machine — is a completely different challenge. This project gives you hands-on experience taking a working multi-agent system and deploying it properly using industry-standard tools: Docker, Kubernetes, and on-premise infrastructure.

What You Will Build

The Application (2 weeks)

Build a simple but functional multi-agent system. Suggested use case: a **customer support routing system** with:

- **Intake Agent** — receives a user complaint or query
- **Classifier Agent** — classifies it as billing / technical / general
- **Responder Agent** — generates a response based on the category

This application will be your "product" that you deploy. It must have a basic API endpoint (FastAPI or Flask) so it can receive requests.

The Deployment Stack (6 weeks)

Component	What to Do
Docker	Write a Dockerfile for the agent application. It must build and run with <code>docker build</code> and <code>docker run</code>
Docker Compose	Write a <code>docker-compose.yml</code> that runs the app plus any dependencies (e.g. a database or Redis) together with one command
Kubernetes	Write Kubernetes YAML files to deploy the app as a proper service — with at least 2 replicas, a service definition, and a config map for environment variables
Logging	Every agent action must be logged with a timestamp, agent name, input, and output. Logs must be visible inside the container and also written to a file
Health Check	The app must have a <code>/health</code> endpoint that returns OK if the system is running

Component	What to Do
Restart Policy	If the app crashes, Kubernetes must automatically restart it
Environment Config	All API keys and settings must be in environment variables — never hardcoded in code

Deliverables

1. Codebase on GitHub

- All code, Dockerfiles, and Kubernetes YAML files in one repository
- Folder structure must be clean and labelled

2. README.md (Step-by-Step Deployment Guide)

Written in plain English. Must include:

- How to install prerequisites (Docker, kubectl, minikube)
- How to build and run with Docker Compose (one command)
- How to deploy on Kubernetes (exact commands, copy-paste ready)
- How to check logs
- How to verify the app is running

3. Demo

A screen recording or live demo showing:

- The app running via Docker Compose
- The app running on Kubernetes with 2 replicas
- A request being sent and a response returned
- Logs being shown in real time
- The app being killed and Kubernetes restarting it automatically

4. Deployment Report (PDF, max 8 pages)

- Architecture diagram showing all components
- What each file does (Dockerfile, docker-compose, each K8s YAML)
- Challenges faced and how you solved them
- What would you add if you had more time

Evaluation Benchmark

Criteria	Target
Docker build succeeds cleanly	Zero errors
Docker Compose brings up full stack	One command, fully working
Kubernetes deployment runs	At least 2 replicas running
Logging works	Every agent action logged with timestamp

Criteria	Target
Health check endpoint works	Returns 200 OK
Auto-restart on crash	Demonstrated in demo
README clarity	A new person should deploy it without asking any questions

PROJECT 3 — Warehouse Box Counting System Using Video

Intern Role: Computer Vision Engineer **Duration:** 2 Months

Problem Statement

Warehouses have racks filled with boxes. Right now, workers manually count how many boxes are in each rack, which is slow and error-prone. The warehouse already has two camera views per rack — a **top-down view** and a **side view**. Using these two video feeds, we want to automatically count how many boxes are in each rack, compare the count against the expected numbers in an Excel sheet, and generate a report showing what matches and what does not.

What You Will Build

A full end-to-end Python application that does the following, in order:

Step 1 — Read Both Videos Load the top-down video and the side-view video for each rack.

Step 2 — Detect and Count Boxes

- From the **top-down video**: count boxes visible from above (gives you rows and columns)
- From the **side view video**: count boxes visible from the side (gives you height / layers)
- Combine both counts to get a final total box count per rack
- You may use any object detection approach — a pre-trained model like YOLOv8 is recommended

Step 3 — Read the Excel Sheet The Excel file will have one row per rack with the expected box count. Read this file and store the expected numbers.

Step 4 — Cross-Check Compare your counted number against the expected number for each rack. Flag any rack where the numbers do not match.

Step 5 — Generate Report Output a final Excel report.

Deliverables

1. Python Codebase on GitHub

- Clean, commented code
- `README.md` with exact steps to run it (what to install, what command to run, where to put the video files)

2. Output Report (Excel File)

The output Excel file must have exactly these columns:

Rack ID	Top View Count	Side View Count	Final Count	Expected Count	Status	Difference
Rack-01	12	9	18	20	MISMATCH	-2
Rack-02	8	8	16	16	MATCH	0

- **Status** must be either MATCH or MISMATCH
- **Difference** = Final Count minus Expected Count
- Rows with MISMATCH must be highlighted in red in the Excel file

3. Summary Sheet (second tab in the same Excel file)

Metric	Value
Total Racks Checked	
Total Matches	
Total Mismatches	
Mismatch Rate (%)	

4. Demo

A short video or screenshots showing:

- Videos being loaded
- Boxes being detected (bounding boxes drawn on screen)
- Final Excel report opened showing matches and mismatches

Evaluation Benchmark

Criteria	Target
Box detection runs on provided videos	No manual intervention needed
Both views are used in the final count	Top + side combined, not just one
Output Excel matches the format above	Exact columns, red highlighting
Summary sheet is generated	Second tab present and correct
Counting accuracy	At least 80% accuracy on provided test videos
Code runs with one command	<code>python main.py</code> or equivalent

PROJECT 4 — Anomaly Detection in IoT Sensor Time Series Data

Intern Role: Data Science / ML Engineer **Duration:** 2 Months

Problem Statement

IoT devices — machines, sensors, equipment on a factory floor — constantly generate readings like temperature, pressure, vibration, and power consumption. Most of the time these readings are normal. But sometimes a reading spikes, drops suddenly, or behaves in an unusual pattern — these are anomalies. If caught early, anomalies can prevent machine failures or accidents. The goal of this project is to take real IoT sensor time series data, automatically detect anomalies in it, and generate a clear report explaining what happened, when it happened, and why it is considered abnormal.

What You Will Build

Step 1 — Get and Prepare the Data Find a publicly available IoT sensor dataset (suggestions: NASA Turbofan dataset, Numenta Anomaly Benchmark, Kaggle IoT sensor datasets, or any real machine sensor CSV). The dataset must have at least one sensor reading over time (timestamps + values). You may use multiple sensors.

Step 2 — Detect Anomalies Apply at least **two different anomaly detection methods** and compare their results:

Method	Description
Statistical (Z-Score or IQR)	Flag points that are far from the mean
Isolation Forest	A machine learning model that finds outliers
(Optional bonus) LSTM Autoencoder	A deep learning approach for sequence anomalies

Step 3 — Generate the Report

Deliverables

1. Python Codebase on GitHub

- Clean code with comments
- `README.md` with steps to reproduce everything from scratch

2. Anomaly Report (PDF)

The PDF report must follow this exact structure:

Section 1 — Dataset Summary

- Where the data came from
- What sensors / readings are in it
- Time range covered
- Total number of data points

Section 2 — Methodology

- Which detection methods were used and why
- How the threshold for anomaly was decided

Section 3 — Anomaly Log Table

Every detected anomaly must appear in a table like this:

Timestamp	Sensor	Observed Value	Expected Range	Anomaly Type	Severity	Explanation
2024-01-03 14:22:00	Temperature	187°C	60–90°C	Spike	High	Value exceeded normal range by 108%. Possible sensor fault or overheating event.

Section 4 — Graphs

For each sensor with anomalies, include:

- A full time series line graph with anomalies marked as red dots
- A zoomed-in graph around each high-severity anomaly showing the before, during, and after

Section 5 — Summary

- Total anomalies detected per method
- Comparison table: which method found more, which had fewer false positives
- Top 3 most critical anomalies and what they likely indicate

3. Interactive HTML Dashboard (Bonus, but strongly encouraged) A simple Plotly or Dash page showing the time series with anomalies you can hover over.

Evaluation Benchmark

Criteria	Target
At least 2 detection methods used	Both implemented and compared
Anomaly log table present	Every anomaly has a timestamp and explanation
Graphs included in report	Time series + zoomed anomaly view
Explanation quality	Each anomaly has a written reason, not just a number
Report is readable by a non-data-scientist	Plain English explanations throughout
Code runs end to end	One script or notebook, no manual steps

PROJECT 5 — AI Agent for Operational Insights from a Large Database

Intern Role: AI / Backend Engineer **Duration:** 2 Months

Problem Statement

Operations teams deal with massive amounts of data — work orders, machine status, inventory levels, maintenance logs, supplier records, and more. This data sits in databases but most people cannot write SQL to query it. They have to ask someone from the data team every time they want an answer, which creates a bottleneck. The goal of this project is to build an AI agent that sits on top of a large operational database, understands natural language questions from the user, queries the database automatically, and returns the answer in plain English plus a visual chart wherever it makes sense.

What You Will Build

Step 1 — Build the Database (3 weeks)

Design and populate a manufacturing / logistics operations database with at least **10 tables** and **at least 100,000 rows per major table** (use synthetic data generation tools like Faker or Mimesis).

Suggested tables:

Table	Description
machines	Machine ID, name, type, location, status
work_orders	Order ID, machine, start time, end time, status
maintenance_logs	Log ID, machine, technician, issue, resolution, date
downtime_events	Event ID, machine, start, end, reason
inventory	Item ID, name, quantity, warehouse location, reorder level
suppliers	Supplier ID, name, contact, lead time, reliability score
purchase_orders	PO ID, supplier, item, quantity, order date, delivery date
employees	Employee ID, name, role, department, shift
shift_logs	Log ID, employee, shift date, start, end, notes
quality_checks	Check ID, machine, product, pass/fail, defect type, date

Step 2 — Build the AI Agent (5 weeks)

The agent must:

- Accept a natural language question from the user via a simple UI or command line
- Convert the question into a correct SQL query
- Run the query on the database
- Return the answer in plain English
- Automatically generate a chart (bar, line, pie) when the answer involves numbers or trends
- Handle follow-up questions in the same session (remember context)

Deliverables

1. Codebase on GitHub

- All database setup scripts included (so anyone can recreate the DB)
- Agent code, UI code, all dependencies in `requirements.txt`
- `README.md` with exact setup and run instructions

2. Working Demo

The agent must correctly answer at least these 10 questions during the demo:

- | # | Sample Question |
|----|--|
| 1 | Which machine had the most downtime last month? |
| 2 | What is the current inventory level for all items below reorder level? |
| 3 | Show me all open work orders older than 7 days |
| 4 | Which supplier has the worst on-time delivery rate? |
| 5 | How many quality check failures happened per machine this quarter? |
| 6 | Which technician resolved the most maintenance issues? |
| 7 | What is the average downtime duration per machine type? |
| 8 | Which warehouse location has the lowest stock levels? |
| 9 | How many work orders were completed vs pending last week? |
| 10 | Show me the trend of defects over the last 6 months |

3. UI (Simple Web App or Streamlit)

The interface must show:

- A text box where the user types their question
- The SQL query that was generated (shown below the question, for transparency)
- The answer in plain English
- A chart when relevant
- A history of the last 5 questions asked in the session

4. Short Report (PDF, max 5 pages)

- Database schema diagram (tables and relationships)
- How the agent works (flowchart: question → SQL → result → output)
- 3 example questions with the generated SQL and output shown
- Limitations — what kinds of questions the agent cannot answer yet

Evaluation Benchmark

Criteria	Target
Database has 10+ tables with relationships	Foreign keys defined, joins work
Each major table has 100k+ rows	Verified with row count
Agent answers all 10 demo questions correctly	8 out of 10 minimum

Criteria	Target
Charts generated where relevant	At least 5 of the 10 answers include a chart
Follow-up question works	Agent remembers context from previous question
SQL query shown to user	Every answer shows the query that produced it
App runs locally with one setup command	No manual DB tweaking needed after setup